

Section 07: Functions **Concept Map** (1 of 2)

The full path through this section - we open here before each concept, then dive in.



Section 07: Functions **Concept Map** (2 of 2)

The full path through this section - we open here before each concept, then dive in.



Functions: Keywords Covered

Section 07: Functions - the keywords in bright green are the new ones we cover here.

Values TrueFalseNone	Classes class	Context Managers with
Logic & Membership andornotinis	Error Handling tryexceptfinallyraise assert	Cleanup & Flow passdel
Conditionals ifelifelse	Modules & Imports importfromas	Generators & Async yieldasyncawait
Loops forwhilebreakcontinue	Scope globalnonlocal	Soft Keywords matchcase_underscoretype
Functions defreturnlambda		

keyword

New in this section

keyword

Covered earlier

keyword

Not covered yet

Functions: What is a **Function**?

What is a function?

A **function** is a **named block of code** you run again and again. **def** defines it; the body runs only when you **call** it with **()**.

WITHOUT a function copy the block every time

CODE

```
print("Hello!")
print("Enjoy Python")

print("Hello!")
print("Enjoy Python")
```

OUTPUT

```
Hello!
Enjoy Python
Hello!
Enjoy Python
```

WITH a function write it once, call it

CODE

```
def greet():
    print("Hello!")
    print("Enjoy Python")

greet()
greet()
```

WHAT RUNS

```
def greet()  no output
greet()      2 lines
greet()      2 lines
the () is what RUNS it
```

Now change the message to **"Welcome!"**?

Edit **every copy**

2 places now, 20 later

Edit **one line** (inside the def)

```
def greet(): ...
```

Functions: Define and Call a Function

A **function** is a named block of code you define once with **def**, then run many times by calling its name with **()**.

#	CONCEPT	SAMPLE CODE
1	Define a function, then call it <code>def</code> <code>()</code> def names a reusable block once; the <code>()</code> after the name is what RUNS the code inside.	<code>def greet():</code>
2	Docstring <code>"""..."""</code> A triple-quoted first line documents the function; <code>__doc__</code> and <code>help()</code> read it back.	<code>"""Print a friendly welcome message."""</code>
3	Define once, call many times Reuse the function instead of copy-pasting the same lines.	<code>divider()</code>

Functions: What is a **Parameter** and an **Argument**?

One is a name, the other is a value

The **parameter** is the name in the **def** line. The **argument** is the value you pass when you **call**. The value travels in under the parameter's name.

Pass NOTHING the parameter has no value

CODE

```
def greet(name):  
    print(f"Hi, {name}!")  
greet()
```

OUTPUT

```
TypeError  
missing name
```

Pass an ARGUMENT one function, any name

CODE

```
def greet(name):  
    print(f"Hi, {name}!")  
greet("Sam")  
student = "Amy"  
greet(student)
```

WHAT TRAVELS IN

```
name parameter  
"Sam" argument  
Hi, Sam!  
Hi, Amy!  
a variable works
```

So how many values do you pass? **One per parameter**

Too few or too many = TypeError

```
greet() / greet("Sam", "Tom")
```

One each, left to right (by position)

```
describe("Sam", "Hyderabad")
```

Functions: One **Argument**

A **parameter** is the name in the function's def line; an **argument** is the value you pass in the call - so one function can work on any value you give it.

#	CONCEPT	SAMPLE CODE
1	Parameter and argument The def name is the parameter; the call value is the argument.	<pre>def greet(name):</pre>
2	Argument from a variable The value you pass can be a variable, not just typed text.	<pre>greet(student)</pre>
3	Positional arguments Values fill parameters by position, left to right.	<pre>describe("Sam", "Hyderabad")</pre>

Functions: **print** vs **return**

Where does the value GO?

print sends the value to the **screen** and hands your code back **None**. **return** hands the value **back to your code**, so you can store it and use it again.

PRINT ONLY you can see it, you cannot use it

```
def show(a, b):  
    print(a + b)  
got = show(2, 3)
```



THE SCREEN SHOWS

5

you can see it



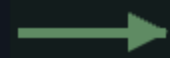
YOUR CODE GETS

None

dead end - nothing to reuse

RETURN the value comes back to your code

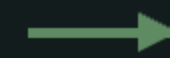
```
def add(a, b):  
    return a + b  
total = add(2, 3)
```



YOUR CODE GETS

5

a real value in total



SO YOU CAN REUSE IT

total * 10 = 50

the value keeps going

print SHOWS, then **hands back None**

```
got = None
```

return hands the value back

```
total = add(2, 3)
```

Functions: print Shows, return Gives Back

return sends a value back from a function so you can store and reuse it, while **print()** only shows a value on screen and hands back None.

#	CONCEPT	SAMPLE CODE
1	print() hands back <code>None</code> You cannot store or reuse what print() shows.	<pre>print(returned) # None</pre>
2	Send a value back <code>return</code> Hands a real value back to store and reuse.	<pre>return a + b</pre>
3	Return a yes/no answer <code>bool</code> Return a comparison to get back True or False.	<pre>return n % 2 == 0</pre>

Functions: Parameters and Logic

A function can take several **parameters** and run real **logic** inside - branches, loops, and early returns - to compute the value it returns.

#	CONCEPT	SAMPLE CODE
1	Multiple parameters Pass several inputs at once; they fill by position.	<pre>def area(width, height):</pre>
2	Logic and return <code>if/elif/else</code> Branch inside the function; return a value per case.	<pre>if marks >= 90:</pre>
3	Loop and return <code>for</code> Loop over items, build a running total, then return it.	<pre>running_total = running_total + number</pre>
4	Return early <code>return</code> Stop when found; the not-found answer comes after the loop.	<pre>return False # only after the loop</pre>

Functions: where does **return False** belong?

One indent level changes the answer

return stops the function immediately. Inside the loop, **return False** decides on the **FIRST** item. After the loop, it decides only once **every** item has been checked.

THE BUG return False inside the loop

```
def has_even_buggy(numbers):  
    for number in numbers:  
        if number % 2 == 0:  
            return True  
        else:  
            return False
```

the band is the loop body : return False is **INSIDE** it

has_even_buggy([1, 3, 4])

1 odd **return False, function ends**

3 never checked

4 never checked (and it is even)

False WRONG

THE FIX return False after the loop

```
def has_even(numbers):  
    for number in numbers:  
        if number % 2 == 0:  
            return True  
    return False  
# only after every item
```

the band is the loop body : return False is **AFTER** it

has_even([1, 3, 4])

1 odd **keep going**

3 odd **keep going**

4 even **return True**

True CORRECT

Functions: **Default Parameters** and **Keyword Arguments**

Default values make an argument optional, and **keyword arguments** let you pass values by name so their order does not matter.

#	CONCEPT	SAMPLE CODE
1	Default value <code>=value</code> Used when the caller skips that argument.	<pre>def greet(name, greeting="Hello"):</pre>
2	Defaults come last A default cannot come before a normal parameter.	<pre>def bad(greeting="Hi", name): # error</pre>
3	Keyword arguments <code>name=value</code> Pass by name, so the order does not matter.	<pre>describe(city="Hyderabad", name="Sam")</pre>
4	Mutable-default trap <code>=None</code> A default list is built once - use None instead.	<pre>def add_item_safe(item, basket=None):</pre>

Functions: why does the default list **remember**?

The default is built **ONCE**, when **def** runs

A default value is created at **definition** time, not on every call. So one `[]` is shared by **every** call forever. Use **None** and build a fresh list **inside** the function.

THE TRAP `basket=[]`

DEF TIME - RUNS ONCE

```
def add_item(item, basket=[]):  
    basket.append(item)  
    return basket
```

ONE list, made now
`[]`

CALL TIME - EVERY CALL SHARES THAT ONE LIST

```
add_item("apple") ['apple']  
add_item("banana") ['apple', 'banana']  
add_item("cherry") ['apple', 'banana', 'cherry']
```

it never clears - it only grows

you can see the shared list

```
add_item.__defaults__ (['apple', 'banana', 'cherry'],)
```

THE FIX `basket=None`

DEF TIME - RUNS ONCE

```
def add_item_safe(item, basket=None):  
    if basket is None:  
        basket = []  
    basket.append(item)  
    return basket
```

call 1
`[]`

call 2
`[]`

call 3
`[]`

CALL TIME - A FRESH LIST EACH TIME

```
add_item_safe("apple") ['apple']  
add_item_safe("banana") ['banana']  
add_item_safe("cherry") ['cherry']
```

every call starts empty

nothing is shared

None cannot hold items, so there is nothing to leak

Functions: how do you return **TWO** values?

You do not. A function always returns **ONE** object.

The **commas** pack your values into a single **tuple** - the brackets are not what does it. Then you choose how to receive that one tuple.

INSIDE THE FUNCTION

```
return min(numbers), max(numbers)
```

the comma builds the tuple



ONE VALUE COMES BACK

(2, 9)

a tuple, len 2

1. Catch it whole

```
result = low_high([4, 9, 2, 7])
```

result (2, 9) one name, the whole tuple

2. Unpack it

```
lo, hi = low_high([4, 9, 2, 7])
```

lo 2 hi 9 two names, one each

The names must match the count `lo, hi, extra = low_high(...)` **ValueError: not enough values to unpack (expected 3, got 2)**

Need three? Exactly the same rule more commas, more names - still **ONE** tuple

```
return min(numbers), max(numbers), len(numbers)
```

```
lo, hi, count = low_high_count([4, 9, 2, 7])
```

→ **(2, 9, 4)**

→ **lo 2 hi 9 count 4**

Functions: Return **Many** Values

A function can hand back several results at once by separating them with commas, and Python packs them into one **tuple** you can **unpack** into separate variables.

#	CONCEPT	SAMPLE CODE
1	Return a tuple <code>a, b</code> Commas pack the values into one tuple.	<pre>return min(numbers), max(numbers)</pre>
2	Unpack into names <code>lo, hi =</code> Catch each returned value in its own variable.	<pre>lo, hi = low_high([4, 9, 2, 7])</pre>
3	Return three values The same trick scales to three or more.	<pre>lo, hi, count = low_high_count([4, 9, 2, 7])</pre>

Functions: ***args** - the problem it solves

You are writing **ONE** function. The caller might have 3 values, or 10, or 40.

A normal **def** locks the count in: an extra value means **editing the function**, a different count simply **crashes**.

THE
CALLER
HAS

THE PROBLEM - A FIXED COUNT

```
def total(a, b, c)
```

Every parameter is typed by hand, so the count is locked in.

CASE 1
3
VALUES

```
def total(a, b, c):  
    return a + b + c
```

```
print(total(1, 2, 3))  
print(total(10, 20, 30, 40))
```

6
TypeError: total() takes 3 positional
arguments but 4 were given

CASE 2
10
VALUES

```
def total(a, b, c, d, e, f, g, h, i, j):  
    return a + b + c + d + e + f + g + h + i + j
```

```
print(total(1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
```

Ten values? Ten parameters - and a ten-term return line. Eleven? Rewrite both, again.

55 only after rewriting the function

THE FIX - ANY COUNT

```
def total(*nums)
```

***args** collects any number of positional values into **ONE** tuple.

The name is yours - ***args** is just the convention; ***nums** works the same.

```
def total(*nums):  
    return sum(nums)
```

```
print(total(1, 2, 3))  
print(total(10, 20, 30, 40))
```

6
100

```
def total(*nums):  
    return sum(nums)
```

```
print(total(1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
```

The **very same def** as Case 1 - not one character changed. It would take 40 just as happily.

55 nothing to rewrite

Functions: ****kwargs** - the problem it solves

One caller wants gift wrap. The next wants a note. The third wants both, and express.

A normal **def** locks in which options exist and in what order: a new one means editing the function, and leaving one out simply **crashes**.

THE
CALLER
WANTS

THE PROBLEM - A FIXED SET

```
def make_order(item, gift_wrap, note)
```

Every option is typed by hand, so the set and the order are locked in.

CASE 1
2
OPTIONS

```
def make_order(item, gift_wrap, note):  
    print("options:", gift_wrap, note)
```

```
make_order("pen", True, "Thanks")  
make_order("pen", True)
```

```
options: True Thanks  
TypeError: make_order() missing 1 required  
positional argument: 'note'
```

CASE 2
4
OPTIONS

```
def make_order(item, gift_wrap, note,  
                express, insured):  
    print("options:", gift_wrap, note, express, insured)  
  
make_order("pen", True, "Hi", False, True)
```

The signature no longer fits one line, and the call is now four bare values. Which one is insured?

```
options: True Hi False True
```

THE FIX - ANY OPTIONS

```
def make_order(item, **options)
```

****kwargs** collects any number of named values into ONE dict.

The name is yours - ****kwargs** is just the convention; ****options** works the same.

```
def make_order(item, **options):  
    print("options:", options)
```

```
make_order("pen", gift_wrap=True, note="Thanks")  
make_order("pen", gift_wrap=True)
```

```
options: {'gift_wrap': True, 'note': 'Thanks'}  
options: {'gift_wrap': True}
```

```
def make_order(item, **options):  
    print("options:", options)
```

```
make_order("pen", gift_wrap=True, note="Hi",  
            express=False, insured=True)
```

The **very same def** as Case 1 - and every value arrives **wearing its own name**.

```
options: {'gift_wrap': True, 'note': 'Hi', 'express': False,  
          'insured': True}
```

Functions: ***args** and ****kwargs** - in the def

Two ways to say "however many you like"

args** gathers any number of **positional** values into a **tuple**. *kwargs** gathers any number of **named** values into a **dict**. Both live in the **def** line, and both mean the caller is not limited to a fixed count.

***args**

any number of positional values
arrive as a tuple

```
# Define Function
def total(*nums):
    return sum(nums)

# Calling Function
print(total(1, 2, 3))
print(total(10, 20, 30, 40))
```

OUTPUT

```
6
100
```

****kwargs**

any number of named values
arrive as a dict

```
# Define Function
def make_order(item, **options):
    print("options:", options)

# Calling Function
make_order("pen", gift_wrap=True, note="Thanks")
make_order("pen", gift_wrap=True)
```

OUTPUT

```
options: {'gift_wrap': True, 'note': 'Thanks'}
options: {'gift_wrap': True}
```

The names are yours - ***args** and ****kwargs** are only a convention. ***nums** and ****info** work exactly the same. It is the star that does the work, not the word.

Functions: * and ** - at the call

Same symbols, opposite direction

In the **def** they GATHER many values into one object. At the **call** they do the reverse: they **SPREAD** one object you already have back into separate arguments. Where the symbol sits is what decides which.

*list

spread a list into positional arguments

```
# Define Function
def total(*nums):
    return sum(nums)

# Calling Function (unpack the list with *)
scores = [5, 10, 15]
print(total(*scores))
```

`total(5, 10, 15)` is what actually runs

OUTPUT

30

**dict

spread a dict into named arguments

```
# Define Function
def make_order(item, **options):
    print("options:", options)

# Calling Function (unpack the dict with **)
extras = {"gift_wrap": True, "note": "Thanks"}
make_order("pen", **extras)
```

`make_order("pen", gift_wrap=True, note="Thanks")` is what actually runs

OUTPUT

options: {'gift_wrap': True, 'note': 'Thanks'}

You already have the values in a list or a dict - * and ** save you unpacking them by hand. The def side gathers; the call side spreads.

Functions: keyword-only * and both together

Two rules about the def line itself

A bare * forces everything after it to be passed by name. And one function can take both - but *args must come before **kwargs.

def f(*, ...)

a bare star makes what follows keyword-only

```
# Define Function
def make_box(*, width, height):
    return width * height

# Calling Function
print(make_box(width=3, height=4))
```

```
print(make_box(3, 4))
TypeError: make_box() takes 0 positional
arguments but 2 were given
```

OUTPUT

12

*args, **kwargs

one function can take both - a tuple and a dict

```
# Define Function
def make_order(*items, **options):
    print("items:", items)
    print("options:", options)

# Calling Function
make_order("pen", "book", gift_wrap=True, note="Thanks")
```

OUTPUT

```
items: ('pen', 'book')
options: {'gift_wrap': True, 'note': 'Thanks'}
```

Order is not a style choice - it is a rule. `make_order(gift_wrap=True, "pen")` is a `SyntaxError`: a keyword argument can never come before a positional one.

Functions: ***args** and ****kwargs**

args** and *kwargs** let one function accept any number of positional and named arguments, gathering the extras into a tuple and a dict.

#	CONCEPT	SAMPLE CODE
1	Collect positional values *args Gathers any number of values into a tuple.	<pre>def total(*nums):</pre>
2	Collect named values **kwargs Gathers name=value inputs into a dict.	<pre>def make_order(item, **options):</pre>
3	Unpack into a call * ** Spread a list or dict straight into a call.	<pre>print(total(*scores))</pre>
4	Keyword-only arguments * A bare * forces callers to pass by name.	<pre>def make_box(*, width, height):</pre>
5	Take both at once *args **kwargs *args (tuple) must come before **kwargs (dict).	<pre>def make_order(*items, **options):</pre>

Functions: Variable Scope (LEGB)

Scope is the region of code where a name is visible, and the LEGB rule sets the order Python searches for it - Local, Enclosing, Global, Built-in.

#	CONCEPT	SAMPLE CODE
1	Local scope A name made inside a function lives only there.	<code>course = "Python"</code>
2	Global scope A top-level name any function can read.	<code>platform = "StackSimplify"</code>
3	Enclosing scope A nested function sees the outer function's names.	<code>print(course, "on", platform)</code>
4	LEGB lookup order Python searches Local, Enclosing, Global, Built-in.	<code>print(x) # Local wins here</code>
5	The global statement <code>global</code> Lets a function change a global value.	<code>global score</code>
6	The nonlocal statement <code>nonlocal</code> Lets an inner function change the enclosing value.	<code>nonlocal score</code>

Functions: scope - where a name lives

Every variable has a SCOPE

A scope is the region of your program where a name exists and can be seen. You write **three** of them: **Local** inside a function, **Enclosing** in the function around it, and **Global** at the top level of the file. One course page, one name at each depth.

L LOCAL

made inside a function
gone the moment it ends

```
def show_course():  
    course = "Python"  
    print("Course:", course)  
  
show_course()
```

OUTPUT

Course: Python

G GLOBAL

made at the top level of the file
any function can READ it

```
platform = "StackSimplify"  
  
def show_platform():  
    print("Platform:", platform)  
  
show_platform()
```

OUTPUT

Platform: StackSimplify

E ENCLOSING

the outer function's variable
read by a function nested inside

```
def show_course_page():  
    course = "Python"  
  
    def show_title():  
        print(course, "on", platform)  
  
    show_title()  
  
show_course_page()
```

OUTPUT

Python on StackSimplify

Look at the last card: **show_title** reaches the enclosing **course** AND the global **platform** in one line. There is a fourth scope you never write - **Built-in**, the names Python ships with like **print** and **len**. So which one wins when the SAME name exists in several? That is **LEGB**, next.

Functions: the LEGB rule - which x does Python use?

Reading a name searches OUTWARD

Python looks for a name Local, then Enclosing, then Global, then Built-in - **the first match wins** and the search stops right there. That is why the very same **print(x)** gives a different answer at each depth.

B BUILT-IN

print, len, sum ...

G GLOBAL

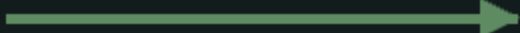
x = "global x"

E ENCLOSING

x = "enclosing x"

L LOCAL

x = "local x"

start where you stand  search outward, first match wins

AND WHEN IT IS IN NONE OF THEM - THE B

```
def show_length():  
    print(len("Python"))  
show_length()
```

Nothing we wrote is called len, so the search runs past L, E and G and finds it in Built-in. Prints 6.

Never shadow one: len = 5 hides the built-in, and len("hi") is broken for the rest of the file.

x = "global x"

```
def legb_outer():  
    x = "enclosing x"  
    def legb_inner():  
        x = "local x"  
        print(x)    1  
    legb_inner()  
    print(x)        2  
legb_outer()  
print(x)           3  
1 local x stops at L  
2 enclosing x legb_outer's own x wins  
3 global x outside both, finds G
```

That is **READING**. Assigning is the opposite - it always creates a **LOCAL** name, which is exactly why **global** and **nonlocal** exist. Next slide.

Functions: **global** and **nonlocal** - writing further out

Assigning inside a function ALWAYS makes a local - unless you say otherwise

So `score = score + 10` reads a name that is already local and still empty: `UnboundLocalError`. One word fixes it, and which word depends only on where the real score lives.

global

the score lives at the top level of the file

WITHOUT global

```
score = 20

def add_points_bad():
    score = score + 10
```

`add_points_bad()` raises
`UnboundLocalError: cannot access local variable 'score' where it is not associated with a value`

WITH global

```
score = 20

def add_points():
    global score
    score = score + 10
    print("Inside the function score:", score)
```

`add_points()` twice
Inside the function score: 30
Inside the function score: 40
Global score: 40

nonlocal

the score lives in the function around it

WITHOUT nonlocal

```
def create_score_bad():
    score = 20
    def add_points():
        score = score + 10
```

`add_points()` raises
`UnboundLocalError: cannot access local variable 'score' where it is not associated with a value`

WITH nonlocal

```
def create_score():
    score = 20
    def add_points():
        nonlocal score
        score = score + 10
        print("Inner Function score:", score)
```

`add_points()` twice
Inner Function score: 30
Inner Function score: 40
Enclosing score: 40

Use these **sparingly** - returning a value is usually cleaner. And that inner `add_points` remembering `score` is a **CLOSURE**: the idea behind decorators.

Functions: **First-class** and Higher-order

A function in Python is itself a **value** you can store and pass around, so a **higher-order function** is simply one that takes or returns another function.

#	CONCEPT	SAMPLE CODE
1	A function is a value Store it in a variable with no parentheses.	<code>yeller = shout # no ()</code>
2	Function as an argument A higher-order function takes another function.	<code>apply_twice(add_three, 10)</code>
3	Transform every item <code>map()</code> Apply one function across a whole list.	<code>list(map(add_three, prices))</code>
4	Keep matching items <code>filter()</code> Keep only the items that pass a test.	<code>list(filter(is_big, prices))</code>
5	Dict dispatch <code>dict</code> Store functions in a dict, pick one by key.	<code>operations["tax"](1000)</code>

Functions: a function is a **VALUE**

One character decides it: the parentheses

The name **without** `()` is the **function itself** - a value you can store, like a number or a string. Add `()` and you get **the result of running it**.

THE FUNCTION WE ARE STORING

```
def shout(text):  
    return text.upper() + "!"
```

NO parentheses shout

```
yeller = shout
```

yeller holds **the function**

type **function**

so you can still call it, through the new name

```
yeller("hello") HELLO!
```

WITH parentheses shout()

```
result = shout("hello")
```

result holds **'HELLO!'**

type **str**

the function is gone - only its answer is left

```
print(result("x")) TypeError: 'str' object is  
not callable
```

A variable can hold a function exactly like it holds anything else:

```
total = 5
```

total holds a number

```
name = "Sam"
```

name holds a string

```
yeller = shout
```

yeller holds a function - same idea

Functions: a function that **takes a function**

The function you pass in becomes **func** inside

A **higher-order** function takes another function as a parameter and calls it. Pass the **name with no ()**, and inside the body **func** is just another name for the function you handed in - so **func(value)** runs it. These three differ in **one thing only**: how many times they call it.

THE FUNCTION WE HAND IN

```
def add_three(n):  
    return n + 3
```

apply_once

```
def apply_once(func, value):  
    return func(value)
```

func is add_three
value is 10

```
apply_once(add_three, 10)  
add_three(10)
```

13 10 TO 13

apply_twice

```
def apply_twice(func, value):  
    return func(func(value))
```

func is add_three
value is 10

```
apply_twice(add_three, 10)  
add_three(add_three(10))
```

16 10 TO 13 TO 16

apply_thrice

```
def apply_thrice(func, value):  
    return func(func(func(value)))
```

func is add_three
value is 10

```
apply_thrice(add_three, 10)  
add_three(add_three(add_three(10)))
```

19 10 TO 13 TO 16 TO 19

add_three never changed, and neither did the 10 we passed in. Only the wrapper did. That is the point of a higher-order function: it does not care which function it receives - pass **shout** from Concept-01 instead and these same three wrappers still work.

Functions: The **lambda** Expression

A **lambda** is a small function with no name, written on one line as **lambda args: expression** and passed straight into another function.

#	CONCEPT	SAMPLE CODE
1	Anonymous function lambda A lambda has no name; to name a function, use def.	<code>(lambda x: x * x)(5)</code>
2	Lambda as a key key= Sort words by length, not alphabetically.	<code>sorted(words, key=lambda word: len(word))</code>
3	Sort by a field pair[1] Reach inside each item to sort by a field.	<code>sorted(pairs, key=lambda pair: pair[1])</code>
4	Transform every item map() A lambda works anywhere a function is expected.	<code>list(map(lambda n: n + 3, prices))</code>
5	Keep matching items filter() A one-line test, written where it is used.	<code>list(filter(lambda n: n > 100, prices))</code>

Functions: **lambda** - a function with no name

One expression, and that expression IS the answer

A **lambda** is a tiny **anonymous** function: no name, no **return** keyword, exactly one expression. Use **def** when the function needs a **name**; use a **lambda** when you are handing it straight to something, right here.

THE FORM **lambda arguments: expression**

ON THIS SLIDE **lambda x: x * x** one argument **x**, and the expression **x * x** IS what comes back

no name

no return

one expression

def

a **NAMED** function you can call again by that name

```
def square_number(x):  
    return x * x  
  
print(square_number(5))
```

OUTPUT

25

lambda

the **SAME** rule, built and called on the spot

```
print((lambda x: x * x)(5))
```

OUTPUT

25

Same answer, **25**. The only difference is what is left behind: **def** leaves a name you can call again, the **lambda** leaves nothing. A **lambda** can take several arguments too: `(lambda x, y: x + y)(2, 3)` is 5.

Functions: `lambda` - as a sort key

The key decides **WHAT** to compare

`sorted(items, key=...)` does not compare the items themselves - it compares whatever the key function returns for each one. Any function will do, named or anonymous.

no key at all `sorted(words)` gives `['apple', 'banana', 'fig', 'kiwi']` alphabetical

`def`

the key is a **NAMED** function, passed by name

```
words = ["banana", "kiwi", "apple", "fig"]

def get_length(word):
    return len(word)

by_length = sorted(words, key=get_length)
print(by_length)
```

OUTPUT

```
['fig', 'kiwi', 'apple', 'banana']
```

`lambda`

the **SAME** key, written inside the call

```
words = ["banana", "kiwi", "apple", "fig"]
by_length = sorted(words, key=lambda word: len(word))
print(by_length)
```

OUTPUT

```
['fig', 'kiwi', 'apple', 'banana']
```

Identical output, shortest first - so pick by whether the key deserves a name. `def` when it is worth naming or reusing; `lambda` when it is this small and used once.

Functions: a key that reaches inside each item

Sort records by a field

When each item is a small record, the key returns the ONE field you want to order by. This is the shape you meet most often in real code.

each item is a pair ("Sam", 70) so pair[0] is the name, pair[1] is the score

def

a NAMED key that picks out the score

```
pairs = [("Sam", 70), ("Tom", 60), ("Ben", 95)]

def get_score(pair):
    return pair[1]

by_score = sorted(pairs, key=get_score)
print(by_score)
```

OUTPUT

```
[('Tom', 60), ('Sam', 70), ('Ben', 95)]
```

lambda

the SAME pick, short enough to read inline

```
pairs = [("Sam", 70), ("Tom", 60), ("Ben", 95)]
by_score = sorted(pairs, key=lambda pair: pair[1])
print(by_score)
```

OUTPUT

```
[('Tom', 60), ('Sam', 70), ('Ben', 95)]
```

Sorted by **SCORE**, not by name - 60, 70, 95. Change pair[1] to pair[0] and the same line sorts by name instead. The key is the whole control.

Functions: when to use a `lambda` - and when not to

The rule of thumb: a lambda is for a function you do not want to name

Both write the **same** rule and both give the **same** answer, so this is never about what is possible; it is about what **reads** better. Ask one question: **does this function deserve a name?**

Reach for `lambda`

- You are handing it straight to another function and nowhere else, like `sorted(words, key=lambda word: len(word))`
- The whole rule is one short expression you can read at a glance, like `lambda pair: pair[1]`
- Naming it would add a line and say nothing that the expression does not already say

Reach for `def`

- You will call it again; a name is the only way to reach it a second time
- It needs more than one expression, or any statement at all, and a lambda cannot hold one
- It is long enough that a good name would explain it, the way `get_score` tells the reader more than the code does

A LAMBDA CANNOT HOLD

`if / elif / else``for``while``try / except``x = 5``return``a docstring`

ALL THREE TAKE A FUNCTION

`sorted(key=...)``map(...)``filter(...)`

you used all three in this step; a lambda fits every one of them

One thing never to do: give a lambda a name. `square_number = lambda x: x * x` takes the one advantage a lambda has (being anonymous) and throws it away, and the error messages get worse too. If it needs a name, it needs `def`.

Functions: Modules and Imports

A **module** is a .py file full of functions, and an **import** pulls that code into your program so you can reuse it.

#	CONCEPT	SAMPLE CODE
1	Import a module <code>import</code> Load a whole module, then call <code>module.function()</code> .	<pre>import math</pre>
2	Import specific names <code>from</code> Pull only the names you need out of a module.	<pre>from math import pi, ceil</pre>
3	Short alias <code>as</code> Give a long module name a short handle.	<pre>import math as m</pre>
4	Import your own file <code>.py</code> Reuse functions you wrote in another .py file.	<pre>from texttools import make_upper</pre>

Functions: what name does **import** put in your file?

The **import** form decides how you must write the call

import math lands the **module**, so you write **math.sqrt()**. **from math import ceil** lands the **name itself**, so you write **ceil()** with no prefix. That is the whole difference.

YOU WRITE	WHAT LANDS IN YOUR FILE	SO YOU CALL IT	GIVES
<code>import math</code>	math the module	<code>math.sqrt(144)</code>	12.0
<code>from math import pi, ceil</code>	pi, ceil the names	<code>ceil(4.1)</code>	5
<code>import math as m</code>	m renamed module	<code>m.sqrt(49)</code>	7.0
<code>from math import factorial as fact</code>	fact renamed name	<code>fact(5)</code>	120

Your own files work exactly the same way: `from texttools import make_upper` lands `make_upper` so `make_upper("welcome")` gives **WELCOME**

as is just a rename; nothing else changes. It is why you will see **import numpy as np** everywhere later. Python finds your own module because it sits in the same folder.

Functions: **recursion** - a function that calls itself

Rule 1 - call yourself on something SMALLER

Every call hands on a smaller job: **countdown(n - 1)**

Rule 2 - a BASE CASE that stops

One value that answers without calling again: **n == 0**

countdown (3)

the work happens on the way DOWN

nothing comes back

CALLS GO DOWN

countdown (3)

```
prints 3
```

countdown (2)

```
prints 2
```

countdown(1)

```
prints 1
```

countdown(0)

```
prints Liftoff!
```

BASE CASE - STOPS HERE

OUTPUT 3 2 1 Liftoff!

factorial(5)

the answer is built on the way **BACK UP**

each return feeds the line above it

VALUES COME BACK UP

```
factorial(5) = 5 * factorial(4)
```

```
factorial(4) = 4 * factorial(3)
```

```
factorial(3) = 3 * factorial(2)
```

```
factorial(2) = 2 * factorial(1)
```

```
factorial(1) = 1
```

BASE CASE - THE RETURNS START HERE

OUTPUT 120

$$5 * 24 = 120$$
$$4 * 6 = 24$$
$$3 * 2 = 6$$
$$2 * 1 = 2$$

No base case, no stop. A recursion that never reaches its base case keeps calling until Python stops it with **RecursionError**, after about 1000 calls. Most everyday repeating is still a loop - recursion is for problems shaped like a smaller version of themselves.

Functions: Recursion

Recursion is when a function **calls itself** to solve a smaller version of the same problem, and a base case stops it.

#	CONCEPT	SAMPLE CODE
1	A function calls itself It repeats by calling itself; a base case stops it.	<code>countdown(n - 1)</code>
2	Factorial, the classic example $n! = n * \text{factorial}(n-1)$, and $\text{factorial}(1)$ is 1.	<code>return n * factorial(n - 1)</code>